

EMBEDDED SYSTEMS INTERNSHIP

WEEK 3 REPORT

Debugging and Performance Optimization

Project: Smart Environment Monitoring and Alert System

Prepared by: Channabasava Kulageri

Platform: ESP32 + Arduino framework

Baseline: DHT22 on GPIO 4

UART/USB: 115200 baud

Validation: deterministic host-side checks + Week 2 simulator evidence

Week 3 focuses on finding faults, fixing them, and demonstrating measurable improvement without changing the intended monitoring behavior.

1. Objective and Scope

The objective of Week 3 is to apply a repeatable debugging process to the Week 2 firmware and then optimize the implementation while preserving the original system requirements. The exercise uses controlled faults so that each defect has a clear symptom, cause, fix, and validation method.

- Reproduce an observable firmware problem before making a fix.
- Use serial logging, source inspection, boundary tests, and deterministic checks to isolate the fault.
- Correct one issue at a time and rerun the same test conditions.
- Measure a concrete performance effect of the optimization.
- Improve the code examples and inline comments so that the debugging decisions are easy to trace.

2. Week 2 Baseline

The Week 2 firmware is the baseline for this exercise. It uses an ESP32, a DHT22 connected on GPIO 4, UART serial at 115200 baud, a nominal monitoring interval of 2000 ms, and configurable warning thresholds. The Week 1 design established the 2-second monitoring target and the GPIO 4 sensor interface.

Parameter	Baseline value	Purpose
Controller	ESP32	Executes firmware
Sensor	DHT22	Temperature + humidity input
Sensor data pin	GPIO 4	Digital sensor interface
UART rate	115200 baud	Debug/status output
Nominal sample interval	2000 ms	Monitoring requirement
Temperature warning	35.0 C	Threshold test point
Humidity warning	80.0 % RH	Threshold test point

3. Debugging Method

The same four-step loop was used for every defect: reproduce -> isolate -> fix -> validate. This avoids changing several variables at once and makes the before/after result traceable.

Stage	Action	Primary evidence	Expected outcome
1. Reproduce	Run the faulty code or deterministic test input.	Serial/log output and test harness	The defect is observable.
2. Isolate	Trace the symptom to timing, boundary logic, or validation.	Source inspection + focused test	Likely fault point is identified.
3. Fix	Change only the defective logic.	Revised source code	Correct behavior without unrelated changes.
4. Validate	Repeat the same inputs and compare results.	Validation log + metrics	Fix passes the intended test.

4. Controlled Bugs Introduced

Three intentionally injected defects represent common embedded-software mistakes. The buggy version is kept separately so the failure can be reproduced without changing the corrected source.

Bug	Injected defect	Expected symptom	Root cause
BUG-01	Sampling interval changed from 2000 ms to 500 ms.	20 scheduled samples occur in 10 s instead of 5.	Timing parameter no longer matches the requirement.
BUG-02	Threshold check changed from <code>>=</code> to <code>></code> .	Exactly 35.0 C or 80.0 % RH stays NORMAL.	Strict comparison excludes the configured boundary.
BUG-03	Invalid-reading check removed.	NaN/invalid data reaches the reporting path.	Firmware assumes every sensor reading is valid.

5. Bug 1 - Excessive Sampling

The first defect violates the established 2-second sampling requirement. A 500 ms interval schedules four times as many sensor cycles as a 2000 ms interval. In a 10-second window the buggy scheduler produces 20 events, while the corrected scheduler produces 5 events.

Buggy code excerpt

```
// BUG: sampling four times faster than the design target.
const unsigned long SAMPLE_INTERVAL_MS = 500;
```

Corrected code excerpt

```
// FIX: restore the Week 1/Week 2 monitoring requirement.
const unsigned long SAMPLE_INTERVAL_MS = 2000;
```

The fixed implementation retains a non-blocking `millis()`-based scheduler. The loop remains available for other work between measurement events, rather than being held inside a long delay.

6. Bug 2 - Threshold Boundary Error

The prototype rule is inclusive: a measurement exactly at the configured boundary is a warning. The faulty code uses a strict greater-than comparison, so a value equal to the boundary is misclassified.

Buggy code excerpt

```
// BUG: 35.0 C does not satisfy 35.0 > 35.0.
if (temperature > TEMP_WARNING_C || humidity > HUM_WARNING_PERCENT) {
    return "WARNING";
}
```

Corrected code excerpt

```
// FIX: inclusive boundary handling.
// Exactly 35.0 C or 80.0 % RH becomes WARNING.
if (temperature >= TEMP_WARNING_C || humidity >= HUM_WARNING_PERCENT) {
    return "WARNING";
}
```

Worked example: at temperature = 35.0 C and humidity = 40.0 %, the buggy expression evaluates to false, so it returns NORMAL. The fixed expression evaluates to true because $35.0 \geq 35.0$, so it returns WARNING.

7. Bug 3 - Invalid Sensor Data Handling

The DHT22 reading functions can return invalid floating-point values. The controlled bug removes the validity gate and immediately uses the returned values in the status/reporting path. The fix checks both values before threshold processing.

Buggy pattern

```
// BUG: no validity check before using the sensor values.
float humidity = dht.readHumidity();
float temperature = dht.readTemperature();
Serial.println(evaluateStatus(temperature, humidity));
```

Corrected code with inline comments

```
// FIX: reject invalid sensor data before threshold evaluation.
const float humidity = dht.readHumidity();
const float temperature = dht.readTemperature();

if (isnan(humidity) || isnan(temperature)) {
    Serial.println("ERROR: Invalid DHT22 reading"); // diagnostic evidence
    return; // retry on next cycle
}
```

The return only ends the current sensor cycle; it does not stop the main monitoring loop. The scheduler therefore remains active and can retry at the next 2000 ms boundary.

8. Code Structure After the Fix

The corrected firmware keeps the control flow simple and separates responsibilities so the fault location is easy to identify. The following excerpt shows the main decision path with inline comments.

Fixed sensor-cycle excerpt

```
void performSensorCycle(unsigned long elapsedMs) {
    sampleCount++;

    // Read the two environmental values from the DHT22.
    const float humidity = dht.readHumidity();
    const float temperature = dht.readTemperature();

    // Stop this cycle if either reading is invalid.
    if (isnan(humidity) || isnan(temperature)) {
        Serial.print("sample=");
        Serial.print(sampleCount);
        Serial.println(" error=INVALID_DHT22_READING action=RETRY_NEXT_CYCLE");
        return;
    }

    // Only valid data reaches threshold processing.
    const char* status = evaluateStatus(temperature, humidity);
    reportMeasurement(elapsedMs, temperature, humidity, status);
}
```

9. Performance Optimization

Two changes were retained in the corrected version. First, the scheduler is restored to 2000 ms, reducing scheduled monitoring events by 75% in the 10-second deterministic comparison. Second, UART output is compacted to one line per valid measurement, with an additional line only when the system state changes. This keeps debugging information while reducing repeated formatting work.

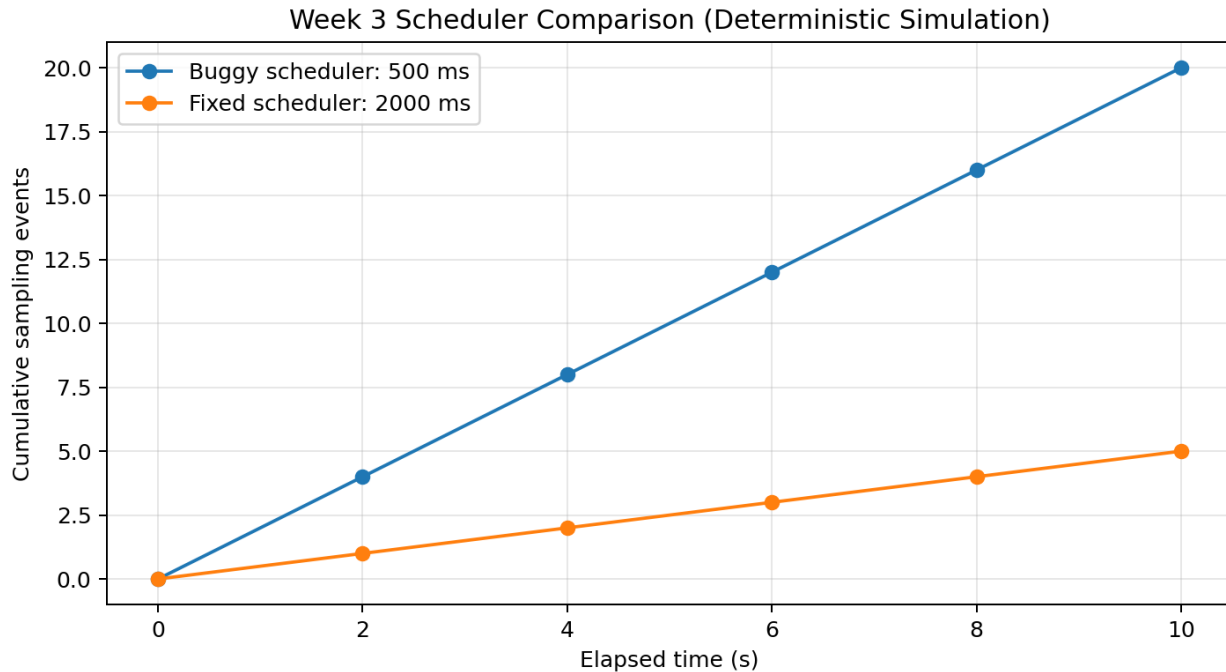


Figure 1. Deterministic scheduler comparison over a 10-second window.

Metric (10 s)	Buggy	Fixed	Interpretation
Scheduled sampling events	20	5	75% fewer events
Nominal interval	500 ms	2000 ms	Meets Week 2 target
35.0 C boundary	NORMAL	WARNING	Boundary corrected
80.0 % RH boundary	NORMAL	WARNING	Boundary corrected
Invalid reading	Unvalidated	Rejected + retry	Fault path protected

The 75% figure is a deterministic scheduling comparison, not an oscilloscope measurement. It is computed from the defined sampling intervals: $10,000 / 500 = 20$ scheduled events and $10,000 / 2,000 = 5$ scheduled events; $(20-5)/20 = 0.75$.

10. Validation Results

The validation harness is a deterministic host-side check of the firmware logic. It verifies scheduling arithmetic, threshold boundaries, invalid-reading handling, and relative monitoring load. It is explicitly presented as supporting logic validation, not as physical-board timing measurement.

Test ID	Input / condition	Expected	Result	Evidence
TIM-01	10 s window	5 fixed events at 2000 ms	PASS	validation_log.txt
THR-01	35.0 C, 40.0 % RH	WARNING	PASS	threshold logic check
THR-02	25.0 C, 80.0 % RH	WARNING	PASS	threshold logic check
ERR-01	NaN temperature	ERROR + retry next cycle	PASS	invalid-reading test
OPT-01	Compare buggy vs fixed sampling count	Fixed < buggy	PASS	20 -> 5 events

11. Simulator Evidence From Week 2

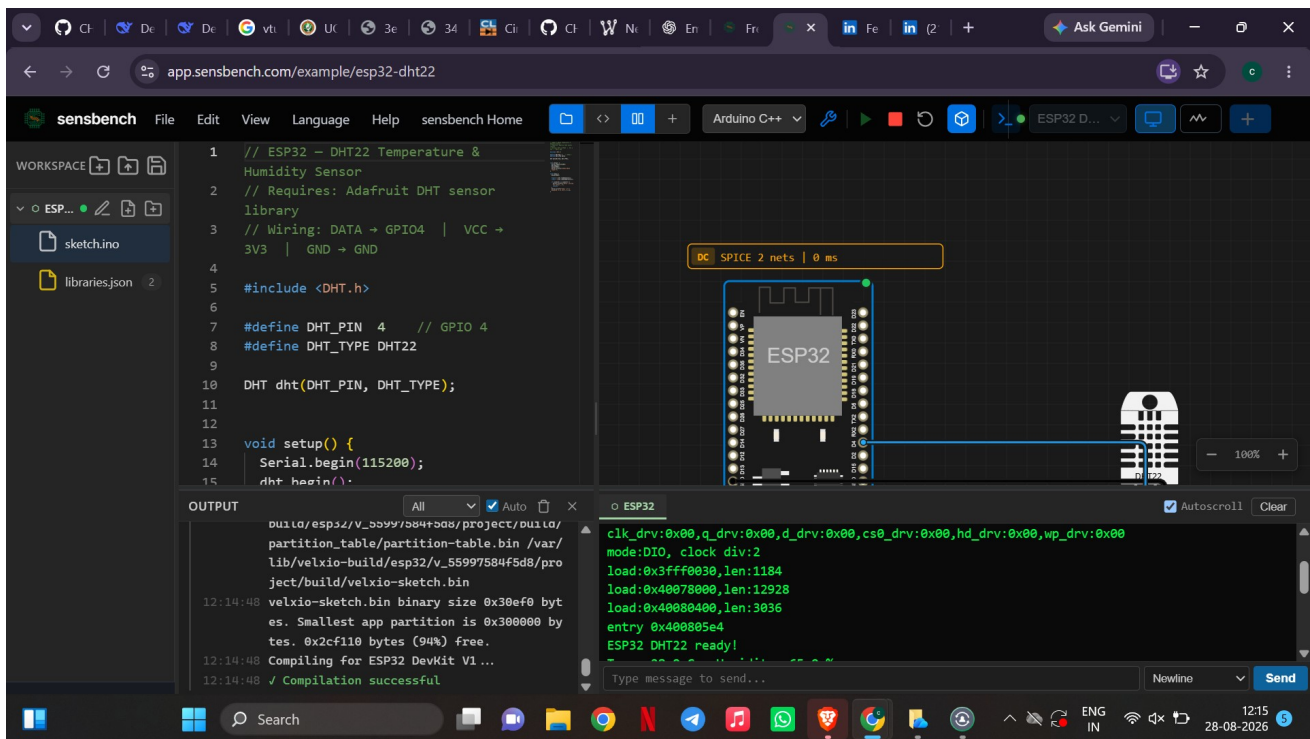


Figure 2. Direct simulator evidence retained from Week 2: successful ESP32 + DHT22 compilation and sensor initialization.

This screenshot is retained as direct simulator evidence for the ESP32/DHT22 environment. The Week 3 deterministic checks are labeled separately so that no host-side result is presented as physical hardware measurement.

12. Traceability to Week 2 Requirements

Week 2 requirement	Week 3 evidence	Status
Periodic monitoring at about 2 s	TIM-01 + corrected scheduler constant Week 3 - Debugging and Performance Optimization	Verified by deterministic check

Sensor validity handling	ERR-01 + isnan() guard	Verified by invalid-input test
Threshold processing	THR-01 and THR-02	Verified at exact boundaries
UART diagnostics	Compact sample/state-change log	Included in fixed firmware
Maintainable firmware structure	Separate status, sensor-cycle, reporting functions	Implemented

13. Debugging Log

Observation: 20 scheduled samples in 10 s at 500 ms

Interpretation: Interpretation: scheduler is four times faster than required.

Action: Action: restore 2000 ms interval.

Observation: 35.0 C remains NORMAL

Interpretation: Interpretation: strict > comparison excludes the boundary.

Action: Action: use >=.

Observation: 80.0 % RH remains NORMAL

Interpretation: Interpretation: same boundary issue on humidity.

Action: Action: use >=.

Observation: NaN can reach reporting path

Interpretation: Interpretation: no input validation before processing.

Action: Action: add isnan() gate.

Observation: Repeated verbose output

Interpretation: Interpretation: unnecessary serial work.

Action: Action: compact record + state-change logging.

14. Limitations and Assumptions

- The 75% performance figure is a deterministic scheduler/load comparison, not an electrical or oscilloscope measurement.
- Direct simulator evidence retained from Week 2 shows successful ESP32 + DHT22 compilation and initialization; Week 3 additional checks use a deterministic host-side harness.
- The optimization targets sampling load and serial-output overhead while preserving the monitoring behavior defined earlier.
- LDR, OLED, buzzer, and Wi-Fi remain future integration items in this project and are not claimed as tested Week 3 hardware.

15. Conclusion

The Week 3 exercise converts the Week 2 firmware into a controlled debugging case study. Three common embedded defects were introduced, isolated, corrected, and validated: an incorrect sampling interval, a threshold-boundary error, and missing invalid-reading protection. The corrected scheduler reduces scheduled monitoring events by 75% in the 10-second deterministic comparison, while the revised threshold logic correctly treats exact boundary values as warnings and the validity gate prevents invalid sensor data from reaching the decision path. The code excerpts include inline comments so the reasoning behind each fix is visible directly in the firmware, addressing the previous feedback requesting stronger code examples and evidence.

Appendix A - Revised Firmware (Key Complete Version)

Complete corrected firmware supplied separately in fixed_version/main_fixed.ino

```
// Smart Environment Monitoring and Alert System - Week 3 FIXED/OPTIMIZED VERSION
// Platform: ESP32 + Arduino framework
// Sensor: DHT22 on GPIO 4
// Communication: UART/USB Serial at 115200 baud
// Week 3 focus: debugging, fault handling, timing correction, and output optimization.

#include <DHT.h>

#define DHT_PIN 4
#define DHT_TYPE DHT22
DHT dht(DHT_PIN, DHT_TYPE);

// Corrected timing requirement from Week 1/2.
const unsigned long SAMPLE_INTERVAL_MS = 2000;
const float TEMP_WARNING_C = 35.0f;
const float HUM_WARNING_PERCENT = 80.0f;

unsigned long lastSampleTime = 0;
unsigned long sampleCount = 0;
const char* lastStatus = "INIT";

const char* evaluateStatus(float temperature, float humidity) {
    // Inclusive boundary handling: exactly 35.0 C or 80.0 % is a warning.
    if (temperature >= TEMP_WARNING_C || humidity >= HUM_WARNING_PERCENT) {
        return "WARNING";
    }
    return "NORMAL";
}

void reportMeasurement(unsigned long elapsedMs,
                      float temperature,
                      float humidity,
                      const char* status) {
    // Compact one-line log reduces serial formatting/output overhead.
    Serial.print("sample=");
    Serial.print(sampleCount);
    Serial.print(" time_ms=");
    Serial.print(elapsedMs);
    Serial.print(" temp_c=");
    Serial.print(temperature, 1);
    Serial.print(" hum_pct=");
    Serial.print(humidity, 1);
    Serial.print(" status=");
    Serial.println(status);

    if (strcmp(status, lastStatus) != 0) {
        Serial.print("STATE_CHANGE: ");
        Serial.println(status);
        lastStatus = status;
    }
}
```

```

void performSensorCycle(unsigned long elapsedMs) {
    sampleCount++;

    const float humidity = dht.readHumidity();
    const float temperature = dht.readTemperature();

    // Fault handling: reject invalid data and continue to the next cycle.
    if (isnan(humidity) || isnan(temperature)) {
        Serial.print("sample=");
        Serial.print(sampleCount);
        Serial.println(" error=INVALID_DHT22_READING action=RETRY_NEXT_CYCLE");
        return;
    }

    const char* status = evaluateStatus(temperature, humidity);
    reportMeasurement(elapsedMs, temperature, humidity, status);
}

void setup() {
    Serial.begin(115200);
    dht.begin();

    Serial.println("Smart Environment Monitor - Week 3 fixed firmware");
    Serial.println("DHT22 GPIO=4 | interval=2000 ms | UART=115200");
    Serial.println("TEMP_LIMIT=35.0 C | HUM_LIMIT=80.0 %");
}

void loop() {
    // Non-blocking scheduler: loop() remains free between measurement events.
    const unsigned long currentTime = millis();

    if (lastSampleTime == 0 ||
        (currentTime - lastSampleTime) >= SAMPLE_INTERVAL_MS) {
        lastSampleTime = currentTime;
        performSensorCycle(currentTime);
    }
}

```

Appendix B - How to Reproduce the Validation

```

# From the Week 3 directory
python validation/validation_sim.py

# Expected final line
All tests passed: True

```

The validation script is intentionally small and deterministic. It can be run on a standard Python installation and does not require physical hardware.

Appendix C - Week 3 Deliverables

- Buggy firmware: buggy_version/main_buggy.ino

EMBEDDED SYSTEMS INTERNSHIP - WEEK 3 | SMART ENVIRONMENT MONITOR

- Fixed/optimized firmware: fixed_version/main_fixed.ino
- Debugging log: debugging_log.md
- Validation script: validation/validation_sim.py
- Validation log: validation/validation_log.txt
- Performance chart: validation/sampling_comparison.png
- Baseline simulation evidence: evidence/week2_simulation.png
- This report: week3_report.docx and week3_report.pdf